

JAVA EXCEPTION HANDLING – 10 PROGRAMS

Complete Practical Record

Each program contains Aim, Algorithm, UML Diagram, Source Code, Execution Commands, Output Screenshot and Result.

Program 1: Array Index Out of Bounds Exception

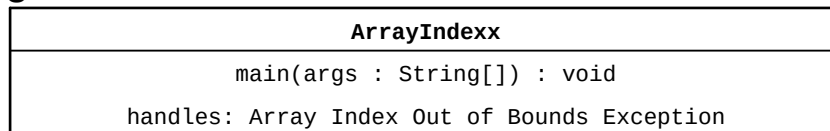
Aim / Objective

To demonstrate exception handling for an invalid array index using `ArrayIndexOutOfBoundsException`.

Algorithm / Logic

1. Create an integer array containing five elements.
2. Use a try block to access an element at index 5.
3. The invalid index causes `ArrayIndexOutOfBoundsException`.
4. Catch the exception.
5. Display "Invalid Array Index".

UML Class Diagram



Source Code

```
public class ArrayIndexx {  
    public static void main(String[] args) {  
        int arr[] = {10,20,30,40,50};  
        try {  
            System.out.println("Element at index 5 : " + arr[5]);  
        }  
        catch(ArrayIndexOutOfBoundsException e){  
  
            System.out.println("Invalid Array Index");  
  
        }  
    }  
}
```

Compilation & Execution Commands

```
javac ArrayIndexx.java  
java ArrayIndexx
```

Output Screenshot

```
PS C:\Users\Student> Invalid Array Index  
The program successfully demonstrates handling of an invalid array index using ArrayIndexOutOfBoundsException.  
PS C:\Users\Student>
```

Result

The program successfully demonstrates handling of an invalid array index using `ArrayIndexOutOfBoundsException`.

Program 2: Array Store Exception

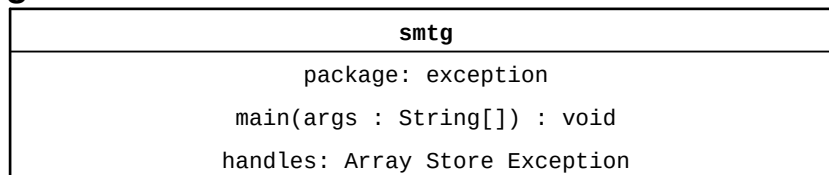
Aim / Objective

To demonstrate exception handling when an incompatible data type is stored in an array using `ArrayStoreException`.

Algorithm / Logic

1. Create an Object array backed by a String array.
2. Store String values in the first two positions.
3. Attempt to store integer 10 in the String array.
4. Catch `ArrayStoreException`.
5. Display the exception-handling message.

UML Class Diagram



Source Code

```
package exception;
public class smtg {
    public static void main(String[] args) {
        Object[] arr = new String[3];
        try {
            arr[0] = "Java";
            arr[1] = "Python";
            arr[2] = 10;
        }
        catch (ArrayStoreException as) {
            System.out.println("Enter same type of data type");
        }
    }
}
```

Compilation & Execution Commands

```
javac -d . smtg.java
java exception.smtg
```

Output Screenshot

```
PS C:\Users\Student> Enter same type of data type
The program successfully demonstrates ArrayStoreException for incompatible data assignment.
PS C:\Users\Student>
```

Result

The program successfully demonstrates `ArrayStoreException` for incompatible data assignment.

Program 3: Class Cast Exception

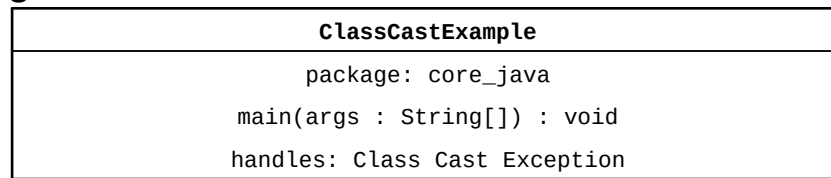
Aim / Objective

To demonstrate handling of `ClassCastException` when an object is cast to an incompatible type.

Algorithm / Logic

1. Create an Object reference containing a String.
2. Attempt to cast the String object to Integer.
3. The incompatible cast throws `ClassCastException`.
4. Catch the exception.
5. Display the exception message.

UML Class Diagram



Source Code

```
package core_java;
public class ClassCastException {
    public static void main(String[] args) {
        try {
            Object obj = "This is a string";
            Integer num = (Integer) obj; // Cannot cast String to Integer
        } catch (ClassCastException e) {
            System.err.println("Caught ClassCastException: Incompatible type casting attempted.");
        }
    }
}
```

Compilation & Execution Commands

```
javac -d . ClassCastException.java
java core_java.ClassCastException
```

Output Screenshot

```
PS C:\Users\Student> Caught ClassCastException: Incompatible type casting attempted.
The program successfully handles an incompatible type cast using ClassCastException.
PS C:\Users\Student>
```

Result

The program successfully handles an incompatible type cast using ClassCastException.

Program 4: File Not Found Exception

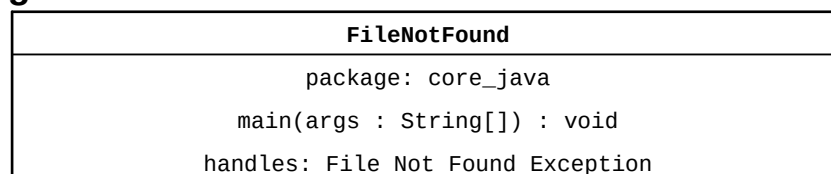
Aim / Objective

To demonstrate handling of FileNotFoundException when a requested file cannot be located.

Algorithm / Logic

1. Create a File object for missing_file.txt.
2. Attempt to create a FileReader for the file.
3. If unavailable, FileNotFoundException is thrown.
4. Catch the exception.
5. Display the file-not-found message.

UML Class Diagram



Source Code

```
package core_java;
import java.io.File;
import java.io.FileReader;
import java.io.FileNotFoundException;

public class FileNotFoundException {
    public static void main(String[] args) {
        try {
            File file = new File("missing_file.txt");
            FileReader fr = new FileReader(file);
        } catch (FileNotFoundException e) {
            System.err.println("Caught FileNotFoundException: File could not be located.");
        }
    }
}
```

```
}  
}
```

Compilation & Execution Commands

```
javac -d . FileNotFound.java  
java core_java.FileNotFound
```

Output Screenshot

```
PS C:\Users\Student> Caught FileNotFoundException: File could not be located.  
The program successfully demonstrates handling of FileNotFoundException for a missing file.  
PS C:\Users\Student>
```

Result

The program successfully demonstrates handling of FileNotFoundException for a missing file.

Program 5: IOException Demo

Aim / Objective

To demonstrate handling of an input/output error using IOException.

Algorithm / Logic

1. Attempt to open student.txt using FileReader.
2. If successful, display the success message and close the file.
3. If an I/O error occurs, catch IOException.
4. Display "Input/Output Error".
5. Display "Program Continues".

UML Class Diagram

IOExceptionDemo
package: exceptionhandling
main(args : String[]) : void
handles: IOException Demo

Source Code

```
package exceptionhandling;  
import java.io.FileReader;  
import java.io.IOException;  
public class IOExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            FileReader file = new FileReader("student.txt");  
            System.out.println("File Opened Successfully");  
            file.close();  
        } catch (IOException e) {  
            System.out.println("Input/Output Error");  
        }  
        System.out.println("Program Continues");  
    }  
}
```

Compilation & Execution Commands

```
javac -d . IOExceptionDemo.java  
java exceptionhandling.IOExceptionDemo
```

Output Screenshot

```
PS C:\Users\Student> Program Continues  
The program demonstrates IOException handling and continues execution after the exception is handled.  
PS C:\Users\Student>
```

Result

The program demonstrates IOException handling and continues execution after the exception is handled.

Program 6: Illegal Argument Exception

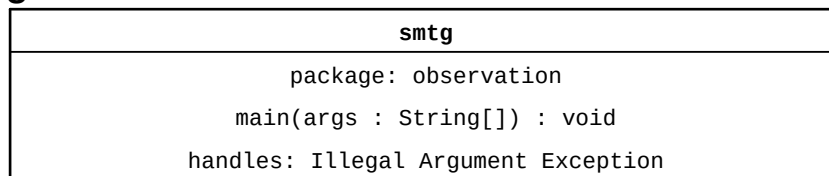
Aim / Objective

To demonstrate throwing and handling `IllegalArgumentException` for an invalid negative volume.

Algorithm / Logic

1. Initialize volume with -10.
2. Check whether volume is less than zero.
3. Throw `IllegalArgumentException` with the given message.
4. Catch the exception.
5. Display the error and reset volume to zero.

UML Class Diagram



Source Code

```
package observation;

public class smtg {
    public static void main(String[] args) {
        int volume = -10;
        try {
            if (volume < 0) {
                throw new IllegalArgumentException("Volume is too low!");
            }
            System.out.println("Volume set to: " + volume);
        } catch (IllegalArgumentException e) {
            System.out.println("Error fixed: " + e.getMessage());
            volume = 0;
            System.out.println("Volume reset to safe level: " + volume);
        }
    }
}
```

Compilation & Execution Commands

```
javac -d . smtg.java
java observation.smtg
```

Output Screenshot

```
PS C:\Users\Student> Volume reset to safe level: 0
The program successfully demonstrates explicitly throwing and handling IllegalArgumentException.
PS C:\Users\Student>
```

Result

The program successfully demonstrates explicitly throwing and handling `IllegalArgumentException`.

Program 7: Input Mismatch Exception

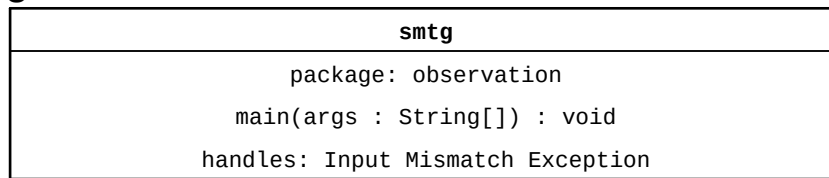
Aim / Objective

To demonstrate handling `InputMismatchException` when the user enters a value that is not an integer.

Algorithm / Logic

1. Create a Scanner object.
2. Prompt the user to enter a number.
3. Read input using `nextInt()`.
4. If input is not an integer, catch `InputMismatchException`.
5. Display the error message and close the Scanner.

UML Class Diagram



Source Code

```
package observation;
import java.util.Scanner;
import java.util.InputMismatchException;
public class smtg {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter a number: ");
            int number = scanner.nextInt(); // Expects a number
            System.out.println("You entered: " + number);
        } catch (InputMismatchException e) {
            System.out.println("That was not a number!");
        }
        scanner.close();
    }
}
```

Compilation & Execution Commands

```
javac -d . smtg.java
java observation.smtg
```

Output Screenshot

```
PS C:\Users\Student> That was not a number!
The program successfully demonstrates handling InputMismatchException for invalid numeric input.
PS C:\Users\Student>
```

Result

The program successfully demonstrates handling InputMismatchException for invalid numeric input.

Program 8: Negative Array Size Exception

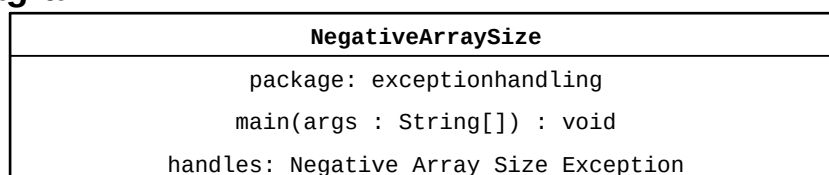
Aim / Objective

To demonstrate handling NegativeArraySizeException when an array is created with a negative size.

Algorithm / Logic

1. Initialize the array size as -2.
2. Attempt to create an integer array with the negative size.
3. The operation throws NegativeArraySizeException.
4. Catch the exception.
5. Display the exception message.

UML Class Diagram



Source Code

```
package exceptionhandling;

public class NegativeArraySize {
    public static void main(String[] args) {
        try {
            int size = -2;
            int[] numbers = new int[size];
        }
    }
}
```

```

        System.out.println("Array created");
    }
    catch (NegativeArraySizeException na) {
        System.out.println("Array size cannot be negative");
    }
}
}

```

Compilation & Execution Commands

```

javac -d . NegativeArraySize.java
java exceptionhandling.NegativeArraySize

```

Output Screenshot

```

PS C:\Users\Student> Array size cannot be negative
The program successfully demonstrates handling NegativeArraySizeException.
PS C:\Users\Student>

```

Result

The program successfully demonstrates handling NegativeArraySizeException.

Program 9: Null Pointer Exception

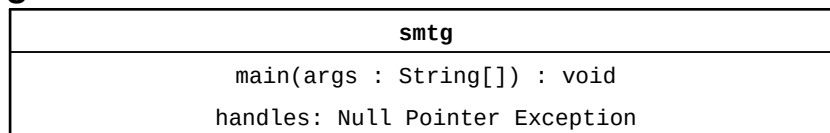
Aim / Objective

To demonstrate handling NullPointerException when a method is called on a null String reference.

Algorithm / Logic

1. Create a String reference and assign null to it.
2. Attempt to call length() on the null reference.
3. Catch the resulting NullPointerException.
4. Display "string is null".

UML Class Diagram



Source Code

```

public class smtg {
    public static void main(String[] args) {
        String name = null;
        try {
            System.out.println("The length of the name is: " + name.length());
        } catch (NullPointerException e) {
            System.out.println("string is null");
        }
    }
}

```

Compilation & Execution Commands

```

javac smtg.java
java smtg

```

Output Screenshot

```

PS C:\Users\Student> string is null
The program successfully demonstrates handling NullPointerException for a null String reference.
PS C:\Users\Student>

```

Result

The program successfully demonstrates handling NullPointerException for a null String reference.

Program 10: String Index Out of Bounds Exception

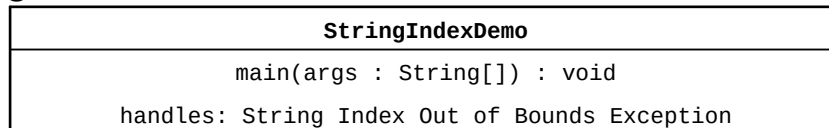
Aim / Objective

To demonstrate handling `StringIndexOutOfBoundsException` when an invalid character index is accessed.

Algorithm / Logic

1. Create the String "Java".
2. Attempt to access character at index 10.
3. The invalid index causes `StringIndexOutOfBoundsException`.
4. Catch the exception.
5. Display the exception message.

UML Class Diagram



Source Code

```
class StringIndexDemo {  
    public static void main(String[] args) {  
        try {  
            String text = "Java";  
            System.out.println(text.charAt(10));  
        } catch (StringIndexOutOfBoundsException e) {  
            System.out.println("Exception: String index is out of bounds");  
        }  
    }  
}
```

Compilation & Execution Commands

```
javac StringIndexDemo.java  
java StringIndexDemo
```

Output Screenshot

```
PS C:\Users\Student> Exception: String index is out of bounds  
The program successfully demonstrates handling StringIndexOutOfBoundsException for an invalid String index.  
PS C:\Users\Student>
```

Result

The program successfully demonstrates handling `StringIndexOutOfBoundsException` for an invalid String index.